

Started on	Thursday, 12 June 2025, 12:46 AM
State	Finished
Completed on	Thursday, 12 June 2025, 12:56 AM
Time taken	9 mins 50 secs
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)



Có n hòn đảo và m cây cầu. Mỗi cây cầu bắt qua 2 hòn đảo. Một hôm chúa đảo tự hỏi là với các cây cầu hiện tại thì đứng ở một hòn đảo bất kỳ có thể nào đi đến được tất cả các hòn đảo khác mà không cần dùng đến thuyền hay không.

Hãy giúp chúa đảo viết chương trình kiểm tra.



Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m, tương ứng là số đảo và số cây cầu.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v nói rằng có 1 cây cầu bắt qua hai hòn đảo u và v.

Đầu ra (Output):

- Nếu có thể đi được in ra màn hình **YES**, ngược lại in ra **NO**.

For example:

Input	Result
4 3 1 2 2 3 3 4	YES

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  typedef int ElementType;
5
6  //-----Graph-----
7  typedef struct {
8      int n, m;
9      int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 int cnt;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     visited[s] = 1;
37     // printf("%d \n", s);
38     for (int u = 1; u <= pG->n; u++) {
39         if (pG->A[s][u] > 0 && !visited[u]) {
40             DFS(pG, u);
41         }
42     }
43 }
44
45
46 int main() {
47     Graph G;

```

```

48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     // DFS(&G, 1);
62
63     cnt = 0;
64     for (int i = 1; i <= G.n; i++) {
65         if (!visited[i]) {
66             cnt++;
67             DFS(&G, i);
68         }
69     }
70     if(cnt>1){
71         printf("NO");
72     }else{
73         printf("YES");
74     }
75 }
76

```

	Input	Expected	Got	
✓	4 3 1 2 2 3 3 4	YES	YES	✓
✓	4 2 3 4 1 2	NO	NO	✓
✓	4 3 2 3 1 4	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Tôn Ngộ Không là một trong các nhân vật chính của truyện Tây du ký. Khi còn ở Hoa Quả Sơn, Tôn Ngộ Không là vua của loài khỉ (Mỹ Hầu Vương). Hoa Quả Sơn có rất nhiều cây ăn trái, nên loài khỉ rất thích. Do đặc tính của mình, khỉ không thích đi mà chỉ thích chuyền từ cây này sang cây khác. Tuy nhiên, nếu khoảng cách giữa hai cây quá xa thì chúng không thể chuyền qua lại được.

Đường đường là vua của loài khỉ, Tôn Ngộ Không muốn vạch ra một kế hoạch hái trái cây trên tất cả các cây có trên Hoa Quả Sơn mà không cần phải nhảy xuống đất. Tôn Ngộ Không dự định sẽ bắt đầu leo lên một cây, hái trái của cây này, sau đó chuyền qua một cây kế tiếp hái trái của cây này và tiếp tục như thế cho đến khi tất cả các cây đều được hái trái. Một cây có thể được chuyền qua nhiều lần.

Hãy giúp Tôn Ngộ Không kiểm tra xem kế hoạch này có thể thực hiện được không.

Đầu vào (Input):

Giả sử số lượng cây ăn trái ở Hoa Quả Sơn là n cây và được đánh số từ 1 đến n .

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số cây và số cặp cây có thể chuyền qua lại.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v , cách nhau 1 khoảng trắng, nói rằng có thể chuyền từ cây u sang cây v hoặc chuyền từ cây v sang cây u .

Đầu ra (Output):

- Nếu kế hoạch của Tôn Ngộ Không có thể thực hiện được **DUOC**, ngược lại in ra **KHONG**.

Xem thêm ví dụ bên dưới. Trong ví dụ đầu tiên, Tôn Ngộ Không bắt đầu từ cây 1, chuyền qua cây 2, sau đó chuyền ngược về 1, chuyền tiếp sang 3 và sau cùng là sang 4.

For example:

Input	Result
4 3 2 1 1 3 3 4	DUOC
4 2 1 2 3 4	KHONG

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  typedef int ElementType;
5
6  //-----Graph-----
7  typedef struct {
8      int n, m;
9      int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----

```

```

33 int cnt;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     visited[s] = 1;
37     // printf("%d \n", s);
38     for (int u = 1; u <= pG->n; u++) {
39         if (pG->A[s][u] > 0 && !visited[u]) {
40             DFS(pG, u);
41         }
42     }
43 }
44 }
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     // DFS(&G, 1);
62
63     cnt = 0;
64     for (int i = 1; i <= G.n; i++) {
65         if (!visited[i]) {
66             cnt++;
67             DFS(&G, i);
68         }
69     }
70     if(cnt>1){
71         printf("KHONG");
72     }else{
73         printf("DUOC");
74     }
75 }
76

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 3 4	DUOC	DUOC	✓
✓	4 2 1 2 3 4	KHONG	KHONG	✓
✓	4 3 1 4 2 3	KHONG	KHONG	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

